Relatório de Auditoria de Segurança de Software

Alvo: Zenite OS (vextron-pdv) | **Metodologia:** Cloudflare Security Audit Skill | **Perfil:** Quick Audit

1. Resumo Executivo

Foi realizada uma auditoria abrangente de segurança e análise de vulnerabilidades no código-fonte do **Zenite OS**. A avaliação identificou **14 apontamentos de segurança**, sendo **1 Crítico**, **9 Altos**, **2 Médios**, **1 Baixo** e **1 item classificado como 'Needs Validation'** (dependente da validação das políticas RLS ativas diretamente no Supabase em produção).

O vetor mais severo consiste em **credenciais administrativas (Gerente) embutidas em texto puro** no bundle público da aplicação frontend (src/App.jsx), realizando login automático para qualquer visitante. Adicionalmente, detectaram-se falhas graves de **isolamento multi-tenant** em RPCs SECURITY DEFINER, no gatilho de cadastro de novos usuários e em lógicas de recuperação de perfil, além de chaves de API expostas no bundle e SSRF no microsserviço fiscal.

Total de Apontamentos	Crítico (Critical)	Alto (High)	Médio (Medium)	Baixo (Low)	Needs Validation
14	1	9	2	1	1

2. Tabela Geral de Vulnerabilidades

Sev.	Identificador / Título	Superfície Afetada	Impacto Resumido
CRITICAL	**Automatic Startup Login with Hardcoded Manager Credentials in Public SPA Bundle** app-jsx-startup-hardcoded-manager-autologin	src/App.jsx	The client bundle exposes both manager email/password pairs in cleartext and automatically authenticates any visitor without `z...
HIGH	**Exposure of Live AI API Keys in Client Bundle and Insecure Fiscal Certificate Password Storage** client-bundle-ai-api-keys-and-static-cert-encryption-key	src/services/geminiService.js	Live Groq, Gemini, and OpenRouter keys are exposed in source and client-side code, and tenant A1 digital certificate passwords ...
HIGH	**Unfiltered Branch Query in Profile Auto-Sync Overwrites User Tenant Binding (`empresa_id`)** dashboard-profile-autoheal-cross-tenant-empresa-overwrite	src/components/Dashboard.jsx	The user's `profileData.empresa_id` in React state, cache, and `public.profiles` is overwritten with the `empresa_id` of the fi...
MEDIUM	**ReportLab XML/HTML Markup Injection in DANFE PDF Generator** fiscal-service/danfe/reportlab-paragraph-xml-injection	fiscal_service/app/routers/nfe.py	ReportLab's XML parser interprets injected tags inside `Paragraph(...)` during `doc.build(elements)` (or raises a parser except...
HIGH	**Fiscal Microservice Default Authentication Bypass and Unvalidated Certificate URL SSRF** fiscal-service/nfe/default-auth-bypass-and-cert-url-ssrf	fiscal_service/app/routers/nfe.py	`verificar_autenticacao` returns `True` without requiring a Bearer token, and `extrair_ou_baixar_certificado` issues an outboun...
HIGH	**Missing-Profile Auto-Recovery Defaults to ADMIN and Escalates Deleted Employees** missing-profile-default-admin-privilege-escalation	src/components/Dashboard.jsx	Instead of being denied access, the deleted employee's profile is recreated with `role: 'ADMIN'` in memory and upserted into `p...
HIGH	**DOM/Stored XSS via Unescaped HTML Interpolation in imprimirRelatorioVendasPDF and Global Error Handlers** spa/dom-xss/relatorio-vendas-document-write-and-error-innerhtml	src/utils/calculoMetas.ts	The popup window opened by `imprimirRelatorioVendasPDF` (or `#root` in `src/main.jsx`) parses the injected HTML tags and execut...
LOW	**Unescaped PostgREST .or() Filter Injection in Customer and Product Lookups** spa/postgrest/dashboard-or-filter-injection	src/components/Dashboard.jsx	The injected predicate is appended to the `or=(...)` query parameter sent to Supabase PostgREST, altering the filter logic of t...
HIGH	**Unauthenticated WhatsApp Webhook Allows Cross-Tenant History Poisoning, LLM Quota Exhaustion, and Outbound Message Spoofing** supabase-edge-functions/whatsapp-webhook-unauthenticated-tenant-injection-and-dispatch	supabase/functions/whatsapp-webhook/index.ts	Source analysis confirms that any unauthenticated POST with `numero_telefone` and `mensagem_texto` writes directly to `whatsapp...
HIGH	**Arbitrary Tenant Attachment via Untrusted User Metadata in SECURITY DEFINER Signup Trigger** supabase/auth-trigger:handle-new-user-untrusted-metadata-tenant-join	src/components/Dashboard.jsx	The `SECURITY DEFINER` trigger `public.handle_new_user()` inserts a `public.profiles` record with `empresa_id = meta_empresa_id...
NEEDS_VALIDATION	**Cross-Tenant and Globally Permissive Row-Level Security Policies Across SQL Migration Scripts** supabase/rls:cross-tenant-permissive-policies-and-admin-bypass	rls_delete_super_admin.sql	Multiple SQL migration scripts in the repository root define Row-Level Security (RLS) policies that break tenant isolati...

HIGH	**Cross-Tenant Data Disclosure and State Modification via Unscoped SECURITY DEFINER RPCs** supabase/rpc:security-definer-cross-tenant-idor-and-modification	src/components/CeoDashboard.jsx	Each `SECURITY DEFINER` RPC executes with owner privileges (bypassing RLS) and reads or mutates records belonging to arbitrary ...
HIGH	**Any Authenticated User Can Mark Arbitrary SaaS Subscription Invoices as Paid via `/api/webhooks/faturas-saas`** vite-middleware/faturas-saas-webhook-missing-authz-cross-tenant-invoice-paid	vite.config.js	Source trace confirms that any authenticated caller can mark their own or another tenant's SaaS subscription invoice as `'PAGO'...
MEDIUM	**Unauthenticated AI Proxy Endpoints Allow Gemini URL Path Injection, API Quota Exhaustion, and Unbounded Body Buffering** vite-middleware/unauthenticated-ai-endpoints-model-path-injection-and-unbounded-body	vite.config.js	Source trace confirms that unauthenticated callers can invoke Gemini and OpenRouter APIs using server credentials, manipulate t...

3. Detalhamento dos Apontamentos Críticos e Altos

Abaixo são apresentados os detalhes técnicos, causas-raízes e recomendações de código para cada vulnerabilidade de impacto severo.

1. [CRITICAL] Automatic Startup Login with Hardcoded Manager Credentials in Public SPA Bundle

Identificador: app-jsx-startup-hardcoded-manager-autologin | **Ponto de Entrada:** src/App.jsx:21

Descrição: The root React component (`src/App.jsx`) embeds plaintext credentials for two branch manager accounts (`rodrigo.gerenciaredecred@gmail.com` and `rodrigo.gerenciamonkeyshop@gmail.com`, both with password `change123`) directly in the client-side bundle and automatically calls `supabase.auth.signInWithPassword` on mount whenever no session is active and `localStorage.getItem('zenite_logged_out') !== 'true'`. Any unauthenticated visitor opening the web application in a fresh browser session is automatically logged into a live `GERENTE` session without providing credentials, and any attacker inspecting the public JavaScript bundle can extract the plaintext credentials directly.

Causa Raiz: Development auto-login logic with hardcoded manager credentials was placed unconditionally inside the production `useEffect` session initializer in `src/App.jsx:20-55`, gated only by a client-side `localStorage` flag (`zenite_logged_out`).

Recomendação de Correção: Remove the automatic sign-in fallback and hardcoded credentials from `src/App.jsx`, leaving `setLoading(false)` when `currentSession` is null so unauthenticated visitors render `<Auth />`. Rotate the passwords for both manager accounts in Supabase Auth.

---

2. [HIGH] Exposure of Live AI API Keys in Client Bundle and Insecure Fiscal Certificate Password Storage

Identificador: client-bundle-ai-api-keys-and-static-cert-encryption-key | **Ponto de Entrada:** src/services/geminiService.js:12

Descrição: Multiple sensitive credentials and cryptographic secrets are exposed in source and the client-side bundle: (1) `.env` commits live API keys (`VITE_GROQ_API_KEY`, `VITE_GEMINI_API_KEY`, `VITE_OPENROUTER_API_KEY`) using the `VITE_` prefix, which Vite inlines into the public browser bundle via `import.meta.env` in `src/services/groqService.js` and `src/services/geminiService.js`; (2) `src/services/geminiService.js:12-15` and `src/services/geminiBackend.cjs:132-135` hardcode a live OpenRouter key (`sk-or-v1-8ba40012...`) split across an array to bypass Git push protection; and (3) A1 digital certificate passwords (`certificado_senha_criptografada`) are encrypted in `vite.config.js:34` using a static fallback AES key (`'VextronLabSaaSKeyDefault32BytesLong'`) when `VITE_SUPABASE_SERVICE_ROLE_KEY` is unset, or stored in raw plaintext by `src/components/Dashboard.jsx:6102-6120` when the local `/api/fiscal/configuracoes` route is unavailable (such as on Vercel static hosting).

Causa Raiz: Secret API keys were configured with Vite's public `VITE_` prefix and hardcoded as array-joined strings in frontend service modules, while fiscal certificate password handling relies on a hardcoded default encryption key in `vite.config.js` and a client-side fallback in `Dashboard.jsx` that writes the unencrypted password directly into `certificado_senha_criptografada`.

Recomendação de Correção: Revoke and rotate all exposed Groq, Gemini, and OpenRouter API keys immediately. Remove `.env` from version control, remove the `VITE_` prefix from all secret API keys, delete the hardcoded `sk-or-v1-...` fallback from `src/services/geminiService.js` and `src/services/geminiBackend.cjs`, require a dedicated high-entropy server-only `CERT_ENCRYPTION_KEY` without a hardcoded fallback or `SUPABASE_SERVICE_ROLE_KEY` reuse, and remove the client-side fallback in `src/components/Dashboard.jsx:6102-6120` that writes plaintext passwords to `configuracoes_fiscais`.

---

3. [HIGH] Unfiltered Branch Query in Profile Auto-Sync Overwrites User Tenant Binding (`empresa_id`)

Identificador: dashboard-profile-autoheal-cross-tenant-empresa-overwrite | **Ponto de Entrada:** src/components/Dashboard.jsx:2650

Descrição: During profile initialization in `fetchProfileAndCompany` (`src/components/Dashboard.jsx:2713-2726`), the client executes `await supabase.from('filiais').select('empresa_id').limit(1)` without any tenant or branch filter. If the returned row's `empresa_id` (`targetEmpId`) differs from `profileData.empresa_id` (or if `profileData.empresa_id` is null), the code immediately overwrites `profileData.empresa_id = targetEmpId` in memory and executes `await supabase.from('profiles').upsert({ id: userId, email: session.user.email, nome: ..., role: userRole, empresa_id: targetEmpId })`. A second unfiltered fallback at lines 2742-2774 repeats this pattern across `filiais` and `companies` when `activeEmpresaId` is unset. Any authenticated user whose profile lacks an `empresa_id` or who can read more than one tenant's branches in `filiais` is forcibly rebound to the arbitrary first `filiais.empresa_id` returned by the query.

Causa Raiz: `fetchProfileAndCompany` in `src/components/Dashboard.jsx:2713-2726` treats the first row of an unfiltered `supabase.from('filiais').select('empresa_id').limit(1)` query as the canonical tenant ID for all users and upserts it over `profileData.empresa_id` whenever they differ.

Recomendação de Correção: Remove the unfiltered `filiais` and `companies` `limit(1)` auto-binding and `profiles` upsert blocks in `fetchProfileAndCompany` (`src/components/Dashboard.jsx:2712-2775`). Rely strictly on the `empresa_id` already stored on the user's `public.profiles` record, and fail closed if a non-`SUPER_ADMIN` profile has no valid `empresa_id`.

---

4. [HIGH] Fiscal Microservice Default Authentication Bypass and Unvalidated Certificate URL SSRF

Identificador: fiscal-service/nfe/default-auth-bypass-and-cert-url-ssrf | **Ponto de Entrada:** fiscal_service/app/routers/nfe.py:23

Descrição: The FastAPI fiscal microservice (`fiscal_service`) defaults `settings.app_secret_token` to `"zenite_fiscal_token"` in `fiscal_service/app/config.py:17`, and `verificar_autenticacao` in `fiscal_service/app/routers/nfe.py:10-20` explicitly skips Bearer token validation whenever `settings.app_secret_token == "zenite_fiscal_token"`. In addition, `fiscal_service/app/main.py:21-27` configures `CORSMiddleware` with `allow_origins=["*"]` and `allow_credentials=True`, and the frontend caller (`src/services/vendasService.js:234-240`) sends requests to `/api/v1/nfe/emitir` without any `Authorization` header. When `/api/v1/nfe/emitir` is invoked, `processar_emissao_nfe_pa` calls `extrair_ou_baixar_certificado(req)` (`fiscal_service/app/services/nfe_service.py:15-39`), which passes `req.certificado_a1_url` directly to `requests.get(req.certificado_a1_url, timeout=15)` without host allowlisting, scheme validation, or private/loopback IP filtering. Any unauthenticated caller or cross-origin browser page that can reach the service can trigger NF-e emission or force the server to perform SSRF requests to internal network or cloud metadata endpoints.

Causa Raiz: `verificar_autenticacao` treats the default secret token `"zenite_fiscal_token"` as a signal to disable authentication (`if settings.app_secret_token and settings.app_secret_token != "zenite_fiscal_token":`), and `extrair_ou_baixar_certificado` performs an unvalidated HTTP GET (`requests.get(req.certificado_a1_url, timeout=15)`) on caller-controlled URLs.

Recomendação de Correção: Fail closed in `verificar_autenticacao` when `APP_SECRET_TOKEN` is missing or equal to the placeholder value, and validate `req.certificado_a1_url` in `extrair_ou_baixar_certificado` so that only HTTPS URLs under the configured `SUPABASE_URL` storage path (`/storage/v1/object/`) are fetched without following redirects.

---

5. [HIGH] Missing-Profile Auto-Recovery Defaults to ADMIN and Escalates Deleted Employees

Identificador: missing-profile-default-admin-privilege-escalation | **Ponto de Entrada:** src/components/Dashboard.jsx:3980

Descrição: When an authenticated user's row in `public.profiles` is absent, both `RoleProtectedRoute` (`src/App.jsx:173-191`) and `fetchProfileAndCompany` (`src/components/Dashboard.jsx:2678-2710`) default the user's role to `'ADMIN'`. Furthermore, `fetchProfileAndCompany` automatically constructs a contingency profile with `role: 'ADMIN'` and persists it to `public.profiles` via `supabase.from('profiles').upsert(profileData)` (`src/components/Dashboard.jsx:2698`). Because employee deletion in `confirmDeleteColaborador` (`src/components/Dashboard.jsx:3965-3993`) and `handleDeleteVendedor` (`src/components/Dashboard.jsx:6998-7010`) only deletes the row in `public.profiles` without deleting or banning the account in `auth.users`, a terminated employee (`VENDEDOR`, `TRAINEE`, or `ESTOQUISTA`) who logs in after being deleted is automatically re-created in `public.profiles` as an `ADMIN`, gaining full administrative UI access and bypassing POS discount restrictions in `src/utils/descontoEngine.js:25-30`.

Causa Raiz: Client-side profile loading fails open by assigning `fallbackRole = 'ADMIN'` (and `data?.role || 'ADMIN'`) and upserting the synthesized `ADMIN` record into `public.profiles` whenever no profile row is found, combined with employee deletion routines that delete only `public.profiles` while leaving `auth.users` credentials active.

Recomendação de Correção: Fail closed when a user's `public.profiles` record is missing or marked `INATIVO`: sign the user out and render an access-denied message instead of defaulting to `'ADMIN'` or upserting a fallback profile from the client. Additionally, perform employee deletion through a privileged server-side endpoint or Edge Function that deletes or bans the user in `auth.users` via `supabaseAdmin.auth.admin.deleteUser(userId)`.

---

6. [HIGH] DOM/Stored XSS via Unescaped HTML Interpolation in imprimirRelatorioVendasPDF and Global Error Handlers

Identificador: spa/dom-xss/relatorio-vendas-document-write-and-error-innerhtml | **Ponto de Entrada:** src/utils/calculoMetas.ts:1668

Descrição: In `src/utils/calculoMetas.ts:1668-1820`, `imprimirRelatorioVendasPDF` builds a complete HTML document string by interpolating `info.produtoNome`, `info.imei`, `info.categoria`, `info.pagamento`, `colaboradorNome`, `competencia`, and `filialNome` without HTML entity encoding, then opens a same-origin blank popup window (`window.open('', '_blank', 'width=1024,height=768')`) and writes the string directly to `win.document.write(html)` (`src/utils/calculoMetas.ts:1817`). Because `about:blank` inherits the opener's origin (`window.origin`), any `<script>` or event handler (such as `<img src=x onerror=...>`) stored in a sale's `produto_nome`, `descricao`, `imei`, `categoria`, `forma_pagamento`, or `filialNome` executes arbitrary JavaScript in the authenticated user's origin (with full access to `localStorage` Supabase session tokens). Additionally, `src/main.jsx:2-26` registers global `error` and `unhandledrejection` listeners that interpolate unescaped `event.message`, `event.reason.message || event.reason`, and stack traces directly into `document.getElementById('root').innerHTML`.

Causa Raiz: `imprimirRelatorioVendasPDF` (`src/utils/calculoMetas.ts:1701-1818`) and the global error handlers (`src/main.jsx:5-24`) interpolate untrusted strings directly into HTML template literals passed to `win.document.write(html)` and `root.innerHTML` without HTML escaping.

Recomendação de Correção: Add an `escapeHtml` helper in `src/utils/calculoMetas.ts` and wrap all dynamic values interpolated into `linhasHtml` and `html`, and replace `root.innerHTML` in `src/main.jsx` with DOM node creation using `textContent`.

---

7. [HIGH] Unauthenticated WhatsApp Webhook Allows Cross-Tenant History Poisoning, LLM Quota Exhaustion, and Outbound Message Spoofing

Identificador: supabase-edge-functions/whatsapp-webhook-unauthenticated-tenant-injection-and-dispatch | **Ponto de Entrada:** supabase/functions/whatsapp-webhook/index.ts:9

Descrição: The `whatsapp-webhook` Supabase Edge Function (`supabase/functions/whatsapp-webhook/index.ts`) accepts unauthenticated HTTP `POST` requests with wildcard CORS (`Access-Control-Allow-Origin: *`) and no provider signature or secret token verification. It initializes a privileged Supabase client using `SUPABASE_SERVICE_ROLE_KEY` (bypassing RLS) and resolves the target tenant directly from caller-supplied `id_loja` / `tenant_id` / `instanceId` fields—or falls back to the first company in `public.companies` when omitted. An unauthenticated attacker can inject fabricated conversation history into `public.whatsapp_triagem_historico` for any tenant, consume the platform's `GROQ_API_KEY` quota, and trigger outbound WhatsApp messages via the victim tenant's configured Evolution API or Z-API gateway (`config.api_url` and `config.api_token`) to arbitrary phone numbers.

Causa Raiz: `supabase/functions/whatsapp-webhook/index.ts` fails to authenticate incoming webhook requests via HMAC signature or shared secret token before using `SUPABASE_SERVICE_ROLE_KEY` to read/write tenant tables (`companies`, `whatsapp_configuracoes`, `whatsapp_triagem_historico`), call Groq completions, and dispatch outbound HTTP requests using stored tenant WhatsApp credentials.

Recomendação de Correção: Require a per-tenant or global cryptographic webhook secret/signature header before processing the webhook payload, remove direct caller selection of arbitrary `tenant_id` without a matching secret bound to `whatsapp_configuracoes`, and remove the fallback that selects the first company from `public.companies`.

---

8. [HIGH] Arbitrary Tenant Attachment via Untrusted User Metadata in SECURITY DEFINER Signup Trigger

Identificador: supabase/auth-trigger:handle-new-user-untrusted-metadata-tenant-join | **Ponto de Entrada:** src/components/Dashboard.jsx:6705

Descrição: The PostgreSQL trigger function `public.handle_new_user()` (`SUPABASE.sql:155-192`), bound to `AFTER INSERT ON auth.users` (`SUPABASE.sql:197-199`), executes as `SECURITY DEFINER` and trusts client-controlled `new.raw_user_meta_data` fields (`role`, `empresa_id`, `filial_id`) supplied during `supabase.auth.signUp()`. Because the frontend provisions employees by calling `supabaseRegister.auth.signUp()` directly from the browser with `options.data: { role: 'VENDEDOR', empresa_id, filial_id }` (`src/components/Dashboard.jsx:3658-3669`, `6705-6716`), any unauthenticated caller who knows or guesses a target tenant's `empresa_id` UUID can self-register via Supabase Auth with `role: 'VENDEDOR'` and that `empresa_id` in `options.data`. The trigger inserts a `public.profiles` row bound to the victim tenant's `empresa_id`, immediately granting the attacker authenticated tenant access (`public.get_user_empresa_id()`) to read and modify the victim company's products, IMEIs, sales, and cash register closures.

Causa Raiz: `public.handle_new_user()` (`SUPABASE.sql:166-176`) reads `role`, `empresa_id`, and `filial_id` directly from unverified client-supplied `new.raw_user_meta_data` inside a `SECURITY DEFINER` trigger on `auth.users` without requiring a server-verified invitation token or service-role provisioning path.

Recomendação de Correção: Remove the `empresa_id` / `filial_id` attachment branch from `public.handle_new_user()` so that public `auth.signUp()` never binds a user to an existing company from `raw_user_meta_data`. Provision tenant employees exclusively through a server-side Edge Function (`supabase/functions/provision-user`) that authenticates the caller's JWT and forces `empresa_id` to match the caller's verified tenant.

---

9. [HIGH] Cross-Tenant Data Disclosure and State Modification via Unscoped SECURITY DEFINER RPCs

Identificador: supabase/rpc:security-definer-cross-tenant-idor-and-modification | **Ponto de Entrada:** src/components/CeoDashboard.jsx:22

Descrição: Six PostgreSQL RPC functions in the repository are declared `SECURITY DEFINER`—bypassing Row-Level Security and executing with the privileges of the function owner—while failing to verify that the target company, sale, or stock transfer belongs to the authenticated caller's tenant (`public.get_user_empresa_id()`): 1. `public.get_ceo_financial_kpis(p_empresa_id UUID)` (`rbac_ceo_security.sql:44-109` and `add_dono_role_and_settings.sql:35-91`, invoked in `src/components/CeoDashboard.jsx:22-24`) verifies only that the caller's role is in `('OWNER', 'DONO', 'SUPER_ADMIN')` (`add_dono_role_and_settings.sql:46-50`) and never checks `p_empresa_id = public.get_user_empresa_id()`. Any tenant's `OWNER` or `DONO` can pass any other tenant's `p_empresa_id` to read its monthly revenue, product cost totals, commissions, gross profit, and branch revenue rankings. 2. `public.corrigir_venda(p_venda_id, p_new_qty, p_new_valor_total, p_new_comissao, p_justificativa)` (`create_sales_audit.sql:91-142`) verifies only that the caller's role is in `('ADMIN', 'SUPER_ADMIN', 'GERENTE', 'OWNER')` (`create_sales_audit.sql:105-108`) without checking that the sale (`id = p_venda_id`) belongs to `public.get_user_empresa_id()`. Any manager or admin of one tenant can overwrite `quantidade`, `valor_total`, and `comissao` on any other tenant's sales. 3. `public.registrar_venda_hibrida(...)` (`SUPABASE.sql:484-640`, invoked in `src/components/Dashboard.jsx:15374-15390`), `public.registrar_transferencia_saida(...)` (`create_transferencias_module.sql:131-175` and `create_estoque_movimentacoes.sql:53-126`, invoked in `src/components/Dashboard.jsx:16540-16547`), `public.confirmar_recebimento_transferencia(p_transferencia_id)` (`create_transferencias_module.sql:179-249` and `create_estoque_movimentacoes.sql:130-221`, invoked in `src/components/Dashboard.jsx:16616-16618`), and `public.registrar_nova_empresa_isolada(...)` (`update_faturamento_metas.sql:17-42`) perform no caller role or tenant ownership check at all, allowing any authenticated user in any tenant to mutate another tenant's inventory, IMEIs, sales, stock transfers, or create arbitrary companies.

Causa Raiz: The `SECURITY DEFINER` functions `get_ceo_financial_kpis`, `corrigir_venda`, `registrar_venda_hibrida`, `registrar_transferencia_saida`, `confirmar_recebimento_transferencia`, and `registrar_nova_empresa_isolada` accept client-supplied tenant and entity identifiers (`p_empresa_id`, `p_venda_id`, `p_transferencia_id`) and execute privileged SQL queries without verifying `p_empresa_id = public.get_user_empresa_id()` (or `public.is_super_admin()`).

Recomendação de Correção: Enforce explicit tenant isolation at the start of every `SECURITY DEFINER` RPC by verifying that the target `empresa_id` (passed directly or looked up from the target `vendas` / `transferencias` row) equals `public.get_user_empresa_id()` unless the caller is verified as `SUPER_ADMIN`, and require `SUPER_ADMIN` in `registrar_nova_empresa_isolada`.

---

10. [HIGH] Any Authenticated User Can Mark Arbitrary SaaS Subscription Invoices as Paid via `/api/webhooks/faturas-saas`

Identificador: vite-middleware/faturas-saas-webhook-missing-authz-cross-tenant-invoice-paid | **Ponto de Entrada:** vite.config.js:196

Descrição: In `vite.config.js` (lines 728-781), the `POST /api/webhooks/faturas-saas` endpoint is grouped under the general authenticated middleware block (lines 194-255) that only verifies a valid Supabase user JWT (`supabase.auth.getUser(token)`). Unlike sibling routes that check `userRole` and `profile?.empresa_id`, `/api/webhooks/faturas-saas` performs no webhook signature verification, no role check (`SUPER_ADMIN`), and no tenant ownership check. It uses the server's `adminClient` (`VITE_SUPABASE_SERVICE_ROLE_KEY`) to look up any `fatura_id` in `public.faturas_saas` and unconditionally updates its `status` to `'PAGO'` along with a generated `nfse_id` and `nfse_pdf_url`.

Causa Raiz: The `/api/webhooks/faturas-saas` handler in `vite.config.js:728-781` executes privileged updates via `adminClient` on `public.faturas_saas` without verifying a payment gateway webhook signature or restricting access to `SUPER_ADMIN`.

Recomendação de Correção: Restrict `/api/webhooks/faturas-saas` to `SUPER_ADMIN` (or verify a dedicated payment gateway HMAC signature header) before querying or updating `public.faturas_saas`.

---

4. Apontamentos Dependentes de Validação (Needs Validation)

[NEEDS VALIDATION] Cross-Tenant and Globally Permissive Row-Level Security Policies Across SQL Migration Scripts

Identificador: supabase/rls:cross-tenant-permissive-policies-and-admin-bypass

Descrição: Multiple SQL migration scripts in the repository root define Row-Level Security (RLS) policies that break tenant isolation across `companies`, `empresas`, `profiles`, `filiais`, `produtos`, `imeis`, `vendas`, `fechamentos`, `produtos_catalogo`, `cores_catalogo`, `clientes`, `metas`, `sales_audit_logs`, and the `fiscal-certificates` Storage bucket: 1. `rls_delete_super_admin.sql:12-21` creates policies `FOR DELETE TO authenticated USING (true)` on both `empresas` and `companies`, allowing any authenticated user in any tenant to delete any company record (which cascade-deletes all branches, products, IMEIs, and sales via `ON DELETE CASCADE` in `SUPABASE.sql:33,211,243,275`). 2. `SUPABASE.sql:79-150,224-333,452-481` and `update_rls_matrix.sql:7-61` include `OR public.get_user_role() = 'ADMIN'` without `empresa_id = public.get_user_empresa_id()`, allowing any tenant user with `role = 'ADMIN'` to read and modify rows across all tenants. 3. `SUPABASE.sql:381-386,655-660`, `rls_clientes_permissao_global.sql:14-31`, and `fix_metas_rls_policies.sql:14-26` create `FOR ALL` / `SELECT` / `INSERT` / `UPDATE` policies `TO authenticated USING (true) WITH CHECK (true)` on `produtos_catalogo`, `cores_catalogo`, `clientes`, and `metas`. 4. `create_sales_audit.sql:78-88` allows any user with role in `('ADMIN', 'SUPER_ADMIN', 'GERENTE', 'OWNER')` to `SELECT` all rows in `public.sales_audit_logs` without tenant filtering, and allows any authenticated user to `INSERT` with `WITH CHECK (true)`. 5. `create_fiscal_bucket.sql:15-29` allows any authenticated user in a tenant (including `VENDEDOR` and `TRAINEE`) to `SELECT` (download) or `INSERT` (upload/overwrite via `upsert: true` in `src/components/Dashboard.jsx:6155-6160`) the company's private A1 `.pfx` certificate in `fiscal-certificates`.

Bloqueador de Validação: Approximately 40 unversioned and mutually conflicting `.sql` scripts exist in the repository root (for example, `create_sales_audit.sql:24-56` defines tenant-scoped policies on `public.clientes` whereas `rls_clientes_permissao_global.sql:14-31` replaces them with `USING (true)`), making the currently active RLS policies in the deployed Supabase PostgreSQL database an unobserved deployment fact.

Ação em Produção: Have the project owner inspect `SELECT schemaname, tablename, policyname, cmd, roles, qual, with_check FROM pg_policies WHERE schemaname IN ('public', 'storage');` in the Supabase SQL Editor to determine which conflicting SQL scripts are currently active in production.

5. Plano de Remediação Recomendado

1. **Imediato (Hotfix Crítico):** Remover o bloco de auto-login e credenciais hardcoded em src/App.jsx.
2. **Rotação de Credenciais:** Rotacionar todas as senhas de contas gerenciais e as chaves de API da Groq, Gemini e OpenRouter expostas no .env.
3. **Isolamento Multi-Tenant:** Atualizar as RPCs SECURITY DEFINER e o gatilho public.handle_new_user() para restringir estritamente qualquer acesso à empresa_id do usuário autenticado.
4. **Microsserviço Fiscal:** Habilitar autenticação Bearer obrigatória no FastAPI e bloquear URLs locais/privadas (SSRF) no download do certificado A1.
5. **Consolidação do Banco:** Organizar os mais de 40 scripts .sql avulsos em migrações versionadas sob supabase/migrations/.